SOURCE CODE

```
#ifndef COMMON_H
#define COMMON_H

#include <string>
#include <iostream>
using namespace std;

const string USERS_FILE =
"data/users.txt";
const string MATCH_REQUESTS_FILE =
"data/match_requests.txt";
const string MATCHES_FILE =
"data/matches.txt";

struct User {
    int id;
    string name;
    string email;
    string phone;
    string password;
    int age;
        int cleanliness;
            int sleepSchedule;
    int studyHabits;
    int socialLevel;
    string description;
    string status;

    User() : id(0), age(0), cleanliness(0),
sleepSchedule(0),
            studyHabits(0), socialLevel(0),
status("PENDING"), description(""),
password("") {}
};

struct MatchRequest {
    int senderId;
    int receiverId;
    string status;
    string senderName;
    string receiverName;

    MatchRequest() : senderId(0),
receiverId(0), status("PENDING") {}
};

struct ConfirmedMatch {
    int user1Id;
    int user2Id;
    string matchDate;

    ConfirmedMatch() : user1Id(0), user2Id(0)
{}
};

string trim(const string& str) {
    if (str.empty()) return str;

    int start = 0;
    int end = str.length() - 1;

    while (start <= end && (str[start] == ' ' ||
str[start] == '\t' || str[start] == '\n')) {
        start++;
    }

    while (end >= start && (str[end] == ' ' ||
str[end] == '\t' || str[end] == '\n')) {
        end--;
    }

    if (start <= end) {
        return str.substr(start, end - start + 1);
    }

    return "";
}

void pause() {
    cout << "\nPress Enter to continue...";
    cin.ignore(10000, '\n');
    cin.get();
}

bool isValidInt(const string& str) {
    if (str.empty()) return false;
    for (int i = 0; i < str.length(); i++) {
        if (i == 0 && str[i] == '-') continue;
        if (!isdigit(str[i])) return false;
    }
    return true;
}

#endif
```

USER_SYSTEM

```
#include <bits/stdc++.h>
#include "common.h"
using namespace std;

class RegistrationModule {
private:
    unordered_map<string, int> eH;
    int nextId;

    void ldU() {
        ifstream file(USERS_FILE);
        if (!file.is_open()) {
            nextId = 1;
            return;
        }

        string line;
        int maxId = 0;

        while (getline(file, line)) {
            if (line.empty()) continue;

            stringstream ss(line);
            string token;
            vector<string> tokens;

            while (getline(ss, token, '|')) {
                tokens.push_back(trim(token));
            }

            if (tokens.size() >= 10) {
                string idStr = tokens[0];
                string email = tokens[2];

                if (isValidInt(idStr)) {
                    int id = stoi(idStr);
                    eH[email] = id;
                    if (id > maxId) maxId = id;
                }
            }
        }

        nextId = maxId + 1;
        file.close();
    }

    bool dupE(const string& email) {
        return eH.find(email) != eH.end();
    }

    bool vE(const string& email) {
        size_t atPos = email.find('@');
        size_t dotPos = email.find('.', atPos);
        return (atPos != string::npos && dotPos
!= string::npos &&
                atPos > 0 && dotPos > atPos + 1);
    }

    bool vP(const string& phone) {
        if (phone.length() != 10) return false;
        for (char c : phone) {
            if (!isdigit(c)) return false;
        }
        return true;
    }

    bool sU(const User& user) {
        ofstream file(USERS_FILE, ios::app);
        if (!file.is_open()) {
            cout << "Error: Could not open users
file!\n";
            cout << "Make sure 'data' folder
exists!\n";
            return false;
        }

        file << user.id << "|"
             << user.name << "|"
             << user.email << "|"
             << user.phone << "|"
             << user.password << "|"
             << user.age << "|"
             << user.cleanliness << "|"
             << user.sleepSchedule << "|"
             << user.studyHabits << "|"
             << user.socialLevel << "|"
             << user.description << "|"
             << user.status << endl;

        file.flush();
        file.close();
        return true;
    }

public:
    RegistrationModule() {
```

```
        ldU();
    }

    void rN() {
        cout <<
"\n================================
================================
================\n";
        cout << "            ROOMMATE
FINDER - NEW USER REGISTRATION\n";
        cout <<
"================================
================================
==============\n\n";

        User newUser;
        newUser.id = nextId;
        newUser.status = "PENDING";

        cout << "Enter your full name: ";
        getline(cin, newUser.name);

        while (true) {
            cout << "Enter your email: ";
            getline(cin, newUser.email);

            if (!vE(newUser.email)) {
                cout << "Invalid email format! Try
again.\n";
                continue;
            }

            if (dupE(newUser.email)) {
                cout << "Email already
registered!\n";
                continue;
            }

            break;
        }

        while (true) {
            cout << "Enter phone number (10
digits): ";
            getline(cin, newUser.phone);

            if (!vP(newUser.phone)) {
                cout << "Invalid! Must be exactly 10
digits.\n";
                continue;
            }
            break;
        }

        while (true) {
            cout << "Create a password (min 6
characters): ";
            getline(cin, newUser.password);

            if (newUser.password.length() < 6) {
                cout << "Password too short! Must
be at least 6 characters.\n";
                continue;
            }
            break;
        }

        while (true) {
            cout << "Enter your age: ";
            string ageStr;
            getline(cin, ageStr);

            if (isValidInt(ageStr)) {
                newUser.age = stoi(ageStr);
                if (newUser.age >= 18 &&
newUser.age <= 35) break;
            }
            cout << "Age must be between 18-
35!\n";
        }

        cout << "\n--- PREFERENCE SCORES
(Rate 1-10) ---\n\n";

        while (true) {
            cout << "Cleanliness (1=messy,
10=very clean): ";
            string val;
            getline(cin, val);
            if (isValidInt(val)) {
                newUser.cleanliness = stoi(val);
                if (newUser.cleanliness >= 1 &&
newUser.cleanliness <= 10) break;
            }
            cout << "Must be between 1-10!\n";
```

```
    }

    while (true) {
        cout << "Sleep schedule (1=early bird,
10=night owl): ";
        string val;
        getline(cin, val);
        if (isValidInt(val)) {
            newUser.sleepSchedule = stoi(val);
            if (newUser.sleepSchedule >= 1 &&
newUser.sleepSchedule <= 10) break;
        }
        cout << "Must be between 1-10!\n";
    }

    while (true) {
        cout << "Study habits (1=quiet,
10=group study): ";
        string val;
        getline(cin, val);
        if (isValidInt(val)) {
            newUser.studyHabits = stoi(val);
            if (newUser.studyHabits >= 1 &&
newUser.studyHabits <= 10) break;
        }
        cout << "Must be between 1-10!\n";
    }

    while (true) {
        cout << "Social level (1=introvert,
10=extrovert): ";
        string val;
        getline(cin, val);
        if (isValidInt(val)) {
            newUser.socialLevel = stoi(val);
            if (newUser.socialLevel >= 1 &&
newUser.socialLevel <= 10) break;
        }
        cout << "Must be between 1-10!\n";
    }

    cout << "\nWould you like to add a short
description about yourself? (y/n): ";
    string choice;
    getline(cin, choice);

    if (choice == "y" || choice == "Y") {
        cout << "Enter description (max 200
chars):\n";
        getline(cin, newUser.description);
        if (newUser.description.length() > 200)
{
            newUser.description =
newUser.description.substr(0, 200);
        }
    } else {
        newUser.description = "No description
provided";
    }

    eH[newUser.email] = newUser.id;

    if (sU(newUser)) {
        cout <<
"\n==============================
==============================
================\n";
        cout << "REGISTRATION
SUCCESSFUL!\n\n";
        cout << "Application ID: " <<
newUser.id << "\n";
        cout << "Status: PENDING\n\n";
        cout << "Your application will be
reviewed by admin.\n";
        cout << "You'll be able to lg once
approved.\n";
        cout <<
"==============================
==============================
================\n";

        nextId++;
    } else {
        cout << "\nRegistration failed!\n";
    }

    pause();
  }
};

struct BSTNode {
  User user;
  BSTNode* left;
  BSTNode* right;
```

```
    BSTNode(User u) : user(u), left(nullptr),
right(nullptr) {}
};

class UserProfileModule {
private:
    User cu;
    bool isLoggedIn;
    BSTNode* userBST;
    vector<User> au;

    void ldUs() {
        au.clear();
        ifstream file(USERS_FILE);
        if (!file.is_open()) return;

        string line;
        while (getline(file, line)) {
            if (line.empty()) continue;

            stringstream ss(line);
            string token;
            vector<string> tokens;

            while (getline(ss, token, '|')) {
                tokens.push_back(trim(token));
            }

            if (tokens.size() >= 12) {
                try {
                    User user;
                    user.id = stoi(tokens[0]);
                    user.name = tokens[1];
                    user.email = tokens[2];
                    user.phone = tokens[3];
                    user.password = tokens[4];
                    user.age = stoi(tokens[5]);
                    user.cleanliness = stoi(tokens[6]);
                    user.sleepSchedule =
stoi(tokens[7]);
                    user.studyHabits =
stoi(tokens[8]);
                    user.socialLevel = stoi(tokens[9]);
                    user.description = tokens[10];
                    user.status = tokens[11];

                    au.push_back(user);
                } catch (...) {
                    continue;
                }
            }
        }

        file.close();
    }

    BSTNode* insertBST(BSTNode* node,
const User& user) {
        if (node == nullptr) {
            return new BSTNode(user);
        }

        if (user.name < node->user.name) {
            node->left = insertBST(node->left,
user);
        } else {
            node->right = insertBST(node->right,
user);
        }

        return node;
    }

    void buildBST() {
        userBST = nullptr;
        for (const User& user : au) {
            if (user.status == "APPROVED" &&
user.id != cu.id) {
                userBST = insertBST(userBST,
user);
            }
        }
    }

    void inorderTraversal(BSTNode* node,
vector<User>& users) {
        if (node == nullptr) return;

        inorderTraversal(node->left, users);
        users.push_back(node->user);
        inorderTraversal(node->right, users);
    }

    void displayUser(const User& user, bool
showContact = false) {
```

```
        cout << "\n----------------------------------------
----------------------------------------\n";
        cout << "ID: " << user.id << " | Name: "
<< user.name << " | Age: " << user.age <<
"\n";

        if (showContact) {
            cout << "Email: " << user.email <<
"\n";
            cout << "Phone: " << user.phone <<
"\n";
        }

        cout << "Preferences: Clean=" <<
user.cleanliness
             << " | Sleep=" << user.sleepSchedule
             << " | Study=" << user.studyHabits
             << " | Social=" << user.socialLevel <<
"\n";

        cout << "Description: " <<
user.description << "\n";
        cout << "----------------------------------------
----------------------------------------\n";
    }

public:
    UserProfileModule() : isLoggedIn(false),
userBST(nullptr) {}

    bool lg() {
        cout <<
"\n================================
================================
================\n";
        cout << "                ROOMMATE
FINDER - USER LOGIN\n";
        cout <<
"================================
================================
================\n\n";

        string email, password;

        cout << "Enter your email: ";
        getline(cin, email);
        email = trim(email);
        transform(email.begin(), email.end(),
email.begin(), ::tolower);

        cout << "Enter your password: ";
        getline(cin, password);
        password = trim(password);

        ldUs();

        for (const User& user : au) {
            string storedEmail = user.email;
            storedEmail = trim(storedEmail);
            transform(storedEmail.begin(),
storedEmail.end(), storedEmail.begin(),
::tolower);

            if (storedEmail == email) {
                if (user.password == password) {
                    if (user.status == "APPROVED")
{
                        cu = user;
                        isLoggedIn = true;
                        cout << "\nLogin
successful!\n";
                        cout << "Welcome, " <<
user.name << "!\n";
                        pause();
                        return true;
                    } else if (user.status ==
"PENDING") {
                        cout << "\nYour application is
pending approval.\n";
                        cout << "Please wait for admin
to review.\n";
                        pause();
                        return false;
                    } else {
                        cout << "\nYour application
was rejected.\n";
                        pause();
                        return false;
                    }
                } else {
                    cout << "\nIncorrect
password!\n";
                    pause();
                    return false;
                }
```

```
        }
      }

      cout << "\nEmail not found! Please
register first.\n";
      pause();
      return false;
   }

   void vMP() {
      cout <<
"\n================================
================================
================\n";
      cout << "                    MY
PROFILE\n";
      cout <<
"================================
================================
================\n";

      displayUser(cu, true);
      pause();
   }

   void browseAllUsers() {
      ldUs();
      buildBST();

      vector<User> sortedUsers;
      inorderTraversal(userBST, sortedUsers);

      if (sortedUsers.empty()) {
         cout << "\nNo other approved users
found.\n";
         pause();
         return;
      }

      cout <<
"\n================================
================================
================\n";
      cout << "      ALL USERS
(Alphabetically Sorted - BST Inorder
Traversal)\n";
      cout <<
"================================
================================
================\n";
      cout << "Total: " << sortedUsers.size()
<< " user(s)\n";

      for (int i = 0; i < sortedUsers.size(); i++)
{
         displayUser(sortedUsers[i], true);

         if ((i + 1) % 5 == 0 && i <
sortedUsers.size() - 1) {
            cout << "\n[Press Enter for
more...]";
            cin.get();
         }
      }

      pause();
   }

   bool getLoginStatus() const { return
isLoggedIn; }
   int getCurrentUserId() const { return cu.id;
}
   User getCurrentUser() const { return cu; }
};

struct Match {
   User user;
   double score;

   bool operator<(const Match& other) const {
      return score < other.score;
   }
};

class MatchingModule {
private:
   vector<User> au;

   void ldUs() {
      au.clear();
      ifstream file(USERS_FILE);
      if (!file.is_open()) {
         cout << "[DEBUG] Could not open
USERS_FILE: " << USERS_FILE << "\n";
         return;
      }
```

```
    string line;
    int lineCount = 0;

    while (getline(file, line)) {
        if (line.empty()) continue;

        lineCount++;

        stringstream ss(line);
        string token;
        vector<string> tokens;

        while (getline(ss, token, '|')) {
            tokens.push_back(trim(token));
        }

        if (tokens.size() >= 12) {
            try {
                User user;
                user.id = stoi(tokens[0]);
                user.name = tokens[1];
                user.email = tokens[2];
                user.phone = tokens[3];
                user.password = tokens[4];
                user.age = stoi(tokens[5]);
                user.cleanliness = stoi(tokens[6]);
                user.sleepSchedule =
stoi(tokens[7]);
                user.studyHabits =
stoi(tokens[8]);
                user.socialLevel = stoi(tokens[9]);
                user.description = tokens[10];
                user.status = tokens[11];

                if (user.status == "APPROVED")
{
                    au.push_back(user);
                }
            } catch (...) {
                cout << "[DEBUG] Error parsing
line " << lineCount << "\n";
                continue;
            }
        } else {
            cout << "[DEBUG] Line " <<
lineCount << " has " << tokens.size()
                 << " tokens (expected 12)\n";
        }
    }

    file.close();
}

double calculateCompatibility(const User&
u1, const User& u2) {
    double score = 0;

    double cleanDiff = abs(u1.cleanliness -
u2.cleanliness);
    score += (10 - cleanDiff) * 2.5;

    double sleepDiff = abs(u1.sleepSchedule
- u2.sleepSchedule);
    score += (10 - sleepDiff) * 2.0;

    double studyDiff = abs(u1.studyHabits -
u2.studyHabits);
    score += (10 - studyDiff) * 2.0;

    double socialDiff = abs(u1.socialLevel -
u2.socialLevel);
    score += (10 - socialDiff) * 3.5;

    return score;
}

void displayMatch(const Match& m, int
rank) {
    cout << "\n----------------------------------
----------------------------------------\n";
    cout << "RANK #" << rank << " -
Compatibility: " << fixed << setprecision(1)
         << m.score << "%\n";
    cout << "ID: " << m.user.id << " | Name:
" << m.user.name << " | Age: " << m.user.age
<< "\n";
    cout << "Email: " << m.user.email <<
"\n";
    cout << "Phone: " << m.user.phone <<
"\n";
    cout << "Preferences: Clean=" <<
m.user.cleanliness
         << " | Sleep=" <<
m.user.sleepSchedule
         << " | Study=" << m.user.studyHabits
```

```
        << " | Social=" << m.user.socialLevel
<< "\n";
        cout << "Description: " <<
m.user.description << "\n";
        cout << "----------------------------------
----------------------------------------\n";
    }

public:

    void findTopMatches(const User& cu) {
        ldUs();

        cout << "\n[DEBUG] Total APPROVED
users loaded: " << au.size() << "\n";
        cout << "[DEBUG] Current user ID: " <<
cu.id << "\n";

        cout << "[DEBUG] Available users:\n";
        for (int i = 0; i < au.size(); i++) {
            cout << "  - ID " << au[i].id << ": " <<
au[i].name
                 << " (status: " << au[i].status <<
")\n";
        }

        priority_queue<Match> maxHeap;

        for (const User& user : au) {
            if (user.id != cu.id) {
                Match m;
                m.user = user;
                m.score = calculateCompatibility(cu,
user);
                maxHeap.push(m);

                cout << "[DEBUG] Added user " <<
user.id << " with score "
                     << fixed << setprecision(1) <<
m.score << "\n";
            }
        }

        cout << "[DEBUG] Total matches in
heap: " << maxHeap.size() << "\n\n";

        if (maxHeap.empty()) {
            cout << "\nNo users available for
matching.\n";
            cout << "Make sure there are other
APPROVED users in the system.\n";
            pause();
            return;
        }

        cout <<
"\n==============================
==================================
================\n";
        cout << "              YOUR TOP
MATCHES (Max Heap - Best First)\n";
        cout <<
"==============================
==================================
================\n";

        int rank = 1;
        int count = 0;

        while (!maxHeap.empty() && count <
10) {
            Match m = maxHeap.top();
            maxHeap.pop();

            displayMatch(m, rank);
            rank++;
            count++;

            if (count % 3 == 0 &&
!maxHeap.empty() && count < 10) {
                cout << "\n[Press Enter for
more...]";
                cin.get();
            }
        }

        pause();
    }

    void sendMatchRequest(int senderId, const
string& senderName) {
        cout << "\n=== SEND MATCH
REQUEST ===\n";

        string idStr;
```

```
        cout << "\nEnter User ID you want to
connect with: ";
        getline(cin, idStr);

        if (!isValidInt(idStr)) {
            cout << "\nInvalid ID!\n";
            pause();
            return;
        }

        int receiverId = stoi(idStr);

        ldUs();
        bool found = false;
        string receiverName;

        for (const User& user : au) {
            if (user.id == receiverId) {
                found = true;
                receiverName = user.name;
                break;
            }
        }

        if (!found) {
            cout << "\nUser ID not found!\n";
            pause();
            return;
        }

        if (receiverId == senderId) {
            cout << "\nCannot send request to
yourself!\n";
            pause();
            return;
        }

        ofstream
file(MATCH_REQUESTS_FILE, ios::app);
        if (!file.is_open()) {
            cout << "Error opening match requests
file!\n";
            pause();
            return;
        }

        file << senderId << "|" << receiverId <<
"|PENDING|"
             << senderName << "|" <<
receiverName << endl;
        file.flush();
        file.close();

        cout << "\nMatch request sent to " <<
receiverName << "!\n";
        cout << "They will be notified about your
interest.\n";
        pause();
    }

    void viewMyRequests(int currentUserId) {
        ifstream
file(MATCH_REQUESTS_FILE);
        if (!file.is_open()) {
            cout << "\nNo match requests file
found.\n";
            pause();
            return;
        }

        vector<MatchRequest> myRequests;
        string line;

        while (getline(file, line)) {
            if (line.empty()) continue;

            stringstream ss(line);
            string token;
            vector<string> tokens;

            while (getline(ss, token, '|')) {
                tokens.push_back(trim(token));
            }

            if (tokens.size() >= 5) {
                try {
                    MatchRequest req;
                    req.senderId = stoi(tokens[0]);
                    req.receiverId = stoi(tokens[1]);
                    req.status = tokens[2];
                    req.senderName = tokens[3];
                    req.receiverName = tokens[4];

                    if (req.receiverId ==
currentUserId) {
                        myRequests.push_back(req);
```

```
            }
        } catch (...) {
            continue;
        }
    }
}

file.close();

if (myRequests.empty()) {
    cout << "\nNo match requests
received.\n";
    pause();
    return;
}

cout <<
"\n================================
================================
================\n";
cout << "                YOUR MATCH
REQUESTS RECEIVED\n";
cout <<
"================================
================================
================\n\n";

for (int i = 0; i < myRequests.size(); i++)
{
    cout << (i+1) << ". From: " <<
myRequests[i].senderName
         << " (ID: " <<
myRequests[i].senderId << ")"
         << " - Status: " <<
myRequests[i].status << "\n";
}

cout <<
"\n================================
================================
================\n";
cout << "\nEnter request number to
respond (0 to go back): ";

string choice;
getline(cin, choice);

if (!isValidInt(choice)) {
    return;
}

int requestNum = stoi(choice);

if (requestNum <= 0 || requestNum >
myRequests.size()) {
    return;
}

MatchRequest selectedRequest =
myRequests[requestNum - 1];

if (selectedRequest.status !=
"PENDING") {
    cout << "\nThis request has already
been " << selectedRequest.status << ".\n";
    pause();
    return;
}

cout << "\n--- RESPOND TO REQUEST
FROM: " << selectedRequest.senderName <<
" ---\n";
cout << "1. Accept\n";
cout << "2. Reject\n";
cout << "3. Cancel\n";
cout << "\nChoice: ";

string action;
getline(cin, action);

if (!isValidInt(action)) return;
int actionChoice = stoi(action);

if (actionChoice == 3) return;
if (actionChoice != 1 && actionChoice !=
2) return;

string newStatus = (actionChoice == 1) ?
"ACCEPTED" : "REJECTED";

ifstream
readFile(MATCH_REQUESTS_FILE);
if (!readFile.is_open()) {
    cout << "\nError reading match
requests file!\n";
    pause();
```

```
            return;
        }

        vector<string> allLines;
        string readLine;

        while (getline(readFile, readLine)) {
            if (readLine.empty()) {
                allLines.push_back(readLine);
                continue;
            }

            stringstream ss(readLine);
            string token;
            vector<string> tokens;

            while (getline(ss, token, '|')) {
                tokens.push_back(trim(token));
            }

            if (tokens.size() >= 5) {
                try {
                    int senderId = stoi(tokens[0]);
                    int receiverId = stoi(tokens[1]);

                    if (senderId ==
selectedRequest.senderId &&
                        receiverId ==
selectedRequest.receiverId &&
                        tokens[2] == "PENDING") {

                        tokens[2] = newStatus;
                    }

                    string updatedLine = "";
                    for (int i = 0; i < tokens.size();
i++) {
                        updatedLine += tokens[i];
                        if (i < tokens.size() - 1)
updatedLine += "|";
                    }
                    allLines.push_back(updatedLine);
                } catch (...) {
                    allLines.push_back(readLine);
                }
            } else {
                allLines.push_back(readLine);
            }
        }

        readFile.close();

        ofstream
writeFile(MATCH_REQUESTS_FILE);
        if (!writeFile.is_open()) {
            cout << "\nError writing to match
requests file!\n";
            pause();
            return;
        }

        for (const string& line : allLines) {
            writeFile << line << "\n";
        }

        writeFile.flush();
        writeFile.close();

        if (actionChoice == 1) {
            time_t now = time(0);
            tm* timeinfo = localtime(&now);
            char buffer[11];
            strftime(buffer, sizeof(buffer), "%Y-
%m-%d", timeinfo);
            string matchDate = string(buffer);

            ofstream matchFile(MATCHES_FILE,
ios::app);
            if (matchFile.is_open()) {
                matchFile <<
selectedRequest.senderId << "|"
                    << selectedRequest.receiverId
<< "|"
                    << matchDate << "\n";
                matchFile.flush();
                matchFile.close();
            }
        }

        cout << "\nRequest " << newStatus <<
"!\n";
        pause();
    }
```

```
    void viewMySentRequests(int
currentUserId, const string&
currentUserName) {
        ifstream
file(MATCH_REQUESTS_FILE);
        if (!file.is_open()) {
            cout << "\nNo match requests file
found.\n";
            pause();
            return;
        }

        vector<MatchRequest> sentRequests;
        string line;

        while (getline(file, line)) {
            if (line.empty()) continue;

            stringstream ss(line);
            string token;
            vector<string> tokens;

            while (getline(ss, token, '|')) {
                tokens.push_back(trim(token));
            }

            if (tokens.size() >= 5) {
                try {
                    MatchRequest req;
                    req.senderId = stoi(tokens[0]);
                    req.receiverId = stoi(tokens[1]);
                    req.status = tokens[2];
                    req.senderName = tokens[3];
                    req.receiverName = tokens[4];

                    if (req.senderId == currentUserId)
{
                        sentRequests.push_back(req);
                    }
                } catch (...) {
                    continue;
                }
            }
        }

        file.close();

        if (sentRequests.empty()) {
            cout << "\nNo match requests sent
yet.\n";
            pause();
            return;
        }

        cout <<
"\n================================
================================
================\n";
        cout << "                YOUR SENT
MATCH REQUESTS\n";
        cout <<
"================================
================================
================\n\n";

        for (int i = 0; i < sentRequests.size(); i++)
{
            cout << (i+1) << ". To: " <<
sentRequests[i].receiverName
                 << " (ID: " <<
sentRequests[i].receiverId << ")"
                 << " - Status: " <<
sentRequests[i].status << "\n";
        }

        cout <<
"\n================================
================================
================\n";
        pause();
    }

    void unmatchUser(int currentUserId, int
matchUserId) {
        ifstream
readFile(MATCH_REQUESTS_FILE);
        if (!readFile.is_open()) {
            cout << "\nError reading match
requests file!\n";
            pause();
            return;
        }

        vector<string> allLines;
        string line;
        bool found = false;
```

```
    while (getline(readFile, line)) {
        if (line.empty()) {
            allLines.push_back(line);
            continue;
        }

        stringstream ss(line);
        string token;
        vector<string> tokens;

        while (getline(ss, token, '|')) {
            tokens.push_back(trim(token));
        }

        if (tokens.size() >= 5) {
            try {
                int senderId = stoi(tokens[0]);
                int receiverId = stoi(tokens[1]);
                string status = tokens[2];

                if (status == "ACCEPTED" &&
                    ((senderId == currentUserId
&& receiverId == matchUserId) ||
                     (senderId == matchUserId &&
receiverId == currentUserId))) {

                    tokens[2] = "UNMATCHED";
                    found = true;
                }

                string updatedLine = "";
                for (int i = 0; i < tokens.size();
i++) {
                    updatedLine += tokens[i];
                    if (i < tokens.size() - 1)
updatedLine += "|";
                }
                allLines.push_back(updatedLine);
            } catch (...) {
                allLines.push_back(line);
            }
        } else {
            allLines.push_back(line);
        }
    }

    readFile.close();

    if (!found) {
        cout << "\nMatch not found!\n";
        pause();
        return;
    }

    ofstream
writeFile(MATCH_REQUESTS_FILE);
    if (!writeFile.is_open()) {
        cout << "\nError writing to match
requests file!\n";
        pause();
        return;
    }

    for (const string& l : allLines) {
        writeFile << l << "\n";
    }

    writeFile.flush();
    writeFile.close();

    cout << "\nMatch removed
successfully!\n";
    pause();
}

void viewConfirmedMatches(int
currentUserId) {
    ifstream
file(MATCH_REQUESTS_FILE);
    if (!file.is_open()) {
        cout << "\nNo match requests file
found.\n";
        pause();
        return;
    }

    vector<MatchRequest>
confirmedMatches;
    string line;

    while (getline(file, line)) {
        if (line.empty()) continue;

        stringstream ss(line);
        string token;
```

```
        vector<string> tokens;

        while (getline(ss, token, '|')) {
            tokens.push_back(trim(token));
        }

        if (tokens.size() >= 5) {
            try {
                MatchRequest req;
                req.senderId = stoi(tokens[0]);
                req.receiverId = stoi(tokens[1]);
                req.status = tokens[2];
                req.senderName = tokens[3];
                req.receiverName = tokens[4];

                if (req.status == "ACCEPTED"
&&
                    (req.receiverId ==
currentUserId || req.senderId ==
currentUserId)) {

confirmedMatches.push_back(req);
                }
            } catch (...) {
                continue;
            }
        }
    }

    file.close();

    if (confirmedMatches.empty()) {
        cout << "\nNo confirmed matches
yet.\n";
        cout << "Keep sending requests to find
your perfect roommate!\n";
        pause();
        return;
    }

    cout <<
"\n================================
================================
================\n";
    cout << "                YOUR
CONFIRMED MATCHES\n";
    cout <<
"================================
================================
================\n\n";
    cout << "Total: " <<
confirmedMatches.size() << " confirmed
match(es)\n\n";

    vector<int> matchIds;
    vector<string> matchNames;

    for (int i = 0; i <
confirmedMatches.size(); i++) {
        string matchName;
        int matchId;

        if (confirmedMatches[i].senderId ==
currentUserId) {
            matchName =
confirmedMatches[i].receiverName;
            matchId =
confirmedMatches[i].receiverId;
        } else {
            matchName =
confirmedMatches[i].senderName;
            matchId =
confirmedMatches[i].senderId;
        }

        matchIds.push_back(matchId);
        matchNames.push_back(matchName);

        cout << (i+1) << ". " << matchName
<< " (ID: " << matchId << ")\n";
    }

    cout <<
"\n================================
================================
================\n";
    cout << "Enter match number (0 to go
back): ";

    string choice;
    getline(cin, choice);

    if (!isValidInt(choice)) {
        return;
    }
```

```
        int matchNum = stoi(choice);

        if (matchNum <= 0 || matchNum >
confirmedMatches.size()) {
            return;
        }

        int selectedMatchId =
matchIds[matchNum - 1];

        ifstream userFile(USERS_FILE);
        if (!userFile.is_open()) {
            cout << "\nError loading user data.\n";
            pause();
            return;
        }

        User matchedUser;
        bool found = false;
        string userLine;

        while (getline(userFile, userLine)) {
            if (userLine.empty()) continue;

            stringstream ss(userLine);
            string token;
            vector<string> tokens;

            while (getline(ss, token, '|')) {
                tokens.push_back(trim(token));
            }

            if (tokens.size() >= 12) {
                try {
                    User user;
                    user.id = stoi(tokens[0]);
                    user.name = tokens[1];
                    user.email = tokens[2];
                    user.phone = tokens[3];
                    user.password = tokens[4];
                    user.age = stoi(tokens[5]);
                    user.cleanliness = stoi(tokens[6]);
                    user.sleepSchedule =
stoi(tokens[7]);
                    user.studyHabits =
stoi(tokens[8]);
                    user.socialLevel = stoi(tokens[9]);
                    user.description = tokens[10];
                    user.status = tokens[11];

                    if (user.id == selectedMatchId) {
                        matchedUser = user;
                        found = true;
                        break;
                    }
                } catch (...) {
                    continue;
                }
            }
        }

        userFile.close();

        if (!found) {
            cout << "\nError loading matched user
profile.\n";
            pause();
            return;
        }

        while (true) {
            cout <<
"\n==============================
==============================
==========\n";
            cout << "                CONFIRMED
MATCH - " << matchedUser.name << "\n";
            cout <<
"==============================
==============================
==========\n\n";

            cout << "1. VIEW FULL
PROFILE\n";
            cout << "2. UNMATCH\n";
            cout << "3. GO BACK\n";
            cout << "\nChoice: ";

            string action;
            getline(cin, action);

            if (!isValidInt(action)) continue;

            int actionChoice = stoi(action);

            if (actionChoice == 1) {
```

```
            cout <<
"\n================================
================================
================\n";
            cout << "                    FULL
PROFILE\n";
            cout <<
"================================
================================
==============\n\n";

            cout << "Name: " <<
matchedUser.name << "\n";
            cout << "ID: " << matchedUser.id
<< "\n";
            cout << "Age: " <<
matchedUser.age << "\n\n";

            cout << "CONTACT
INFORMATION:\n";
            cout << "Email: " <<
matchedUser.email << "\n";
            cout << "Phone: " <<
matchedUser.phone << "\n\n";

            cout << "PREFERENCES:\n";
            cout << "Cleanliness:    " <<
matchedUser.cleanliness << "/10\n";
            cout << "Sleep Schedule: " <<
matchedUser.sleepSchedule << "/10 (1=early
bird, 10=night owl)\n";
            cout << "Study Habits:   " <<
matchedUser.studyHabits << "/10 (1=quiet,
10=group study)\n";
            cout << "Social Level:   " <<
matchedUser.socialLevel << "/10 (1=introvert,
10=extrovert)\n\n";

            cout << "ABOUT THEM:\n";
            cout << matchedUser.description <<
"\n";

            cout <<
"\n================================
================================
================\n";
            cout << "Press Enter to continue...";
            cin.get();

        } else if (actionChoice == 2) {
            cout << "\nUnmatch from " <<
matchedUser.name << "? (y/n): ";
            string confirm;
            getline(cin, confirm);

            if (confirm == "y" || confirm == "Y")
{
                unmatchUser(currentUserId,
selectedMatchId);
                cout << "\nReturning to match
list...\n";
                pause();
                return;
            }

        } else if (actionChoice == 3) {
            return;
        }
    }
};

int main() {
    RegistrationModule registration;
    UserProfileModule userProfile;
    MatchingModule matching;

    while (true) {
        cout <<
"\n================================
================================
================\n";
        cout << "               ROOMMATE
FINDER - STUDENT PORTAL\n";
        cout <<
"================================
================================
==============\n\n";

        if (!userProfile.getLoginStatus()) {
            cout << "1. Register New Account\n";
            cout << "2. Login to Existing
Account\n";
            cout << "3. Exit\n";
            cout << "\nChoice: ";
```

```
            string choiceStr;
            getline(cin, choiceStr);

            if (!isValidInt(choiceStr)) continue;
            int choice = stoi(choiceStr);

            if (choice == 1) {
                registration.rN();
            } else if (choice == 2) {
                userProfile.lg();
            } else if (choice == 3) {
                cout << "\nThank you for using
Roommate Finder!\n";
                break;
            }
        } else {
            cout << "1. View My Profile\n";
            cout << "2. Browse All Users (BST -
Alphabetically)\n";
            cout << "3. Find My Top Matches
(Heap - Best First)\n";
            cout << "4. Send Match Request\n";
            cout << "5. View Received Match
Requests\n";
            cout << "6. View My Sent
Requests\n";
            cout << "7. View Confirmed
Matches\n";
            cout << "8. Logout\n";
            cout << "\nChoice: ";

            string choiceStr;
            getline(cin, choiceStr);

            if (!isValidInt(choiceStr)) continue;
            int choice = stoi(choiceStr);

            switch(choice) {
                case 1:
                    userProfile.vMP();
                    break;
                case 2:
                    userProfile.browseAllUsers();
                    break;
                case 3:

matching.findTopMatches(userProfile.getCurr
entUser());
                    break;
                case 4:

matching.sendMatchRequest(userProfile.getC
urrentUserId(),

userProfile.getCurrentUser().name);
                    break;
                case 5:

matching.viewMyRequests(userProfile.getCur
rentUserId());
                    break;
                case 6:

matching.viewMySentRequests(userProfile.ge
tCurrentUserId(),

userProfile.getCurrentUser().name);
                    break;
                case 7:

matching.viewConfirmedMatches(userProfile.
getCurrentUserId());
                    break;
                case 8:
                    cout << "\nLogged out
successfully!\n";
                    pause();
                    return 0;
            }
        }
    }

    return 0;
}
```

ADMIN_SYSTEM

```
#include <bits/stdc++.h>
#include "common.h"

using namespace std;


const string ADMIN_USERNAME =
"admin";
const string ADMIN_PASSWORD =
"admin123";



class AdminApprovalModule {
private:
   queue<User> pendingQueue;
   vector<User> allUsers;

   void ldUs() {
      allUsers.clear();
      ifstream file(USERS_FILE);
      if (!file.is_open()) {
         cout << "Warning: Users file not
found. Creating new file.\n";
         ofstream newFile(USERS_FILE);
         newFile.close();
         return;
      }

      string line;
      while (getline(file, line)) {
         if (line.empty()) continue;

         stringstream ss(line);
         string token;
                vector<string> tokens;

         while (getline(ss, token, '|')) {
            tokens.push_back(trim(token));
         }

         if (tokens.size() >= 12) {
            try {
               User user;
               user.id = stoi(tokens[0]);
               user.name = tokens[1];
               user.email = tokens[2];
               user.phone = tokens[3];
               user.password = tokens[4];
               user.age = stoi(tokens[5]);
               user.cleanliness = stoi(tokens[6]);
               user.sleepSchedule =
stoi(tokens[7]);
               user.studyHabits =
stoi(tokens[8]);
               user.socialLevel = stoi(tokens[9]);
               user.description = tokens[10];
               user.status = tokens[11];

               allUsers.push_back(user);
            } catch (...) {
               continue;
            }
         }
      }

      file.close();
   }

   void bPQ() {
      while (!pendingQueue.empty()) {
         pendingQueue.pop();
      }

      for (const User& user : allUsers) {
         if (user.status == "PENDING") {
            pendingQueue.push(user);
         }
      }
   }

   bool uUS(int userId, const string&
newStatus) {
      bool updated = false;
      for (User& user : allUsers) {
         if (user.id == userId) {
            user.status = newStatus;
            updated = true;
            break;
         }
      }

      if (!updated) return false;

      ofstream file(USERS_FILE);
```

```
        if (!file.is_open()) {
            cout << "Error: Cannot update users
file!\n";
            return false;
        }

        for (const User& user : allUsers) {
            file << user.id << "|"
                 << user.name << "|"
                 << user.email << "|"
                 << user.phone << "|"
                 << user.password << "|"
                 << user.age << "|"
                 << user.cleanliness << "|"
                 << user.sleepSchedule << "|"
                 << user.studyHabits << "|"
                 << user.socialLevel << "|"
                 << user.description << "|"
                 << user.status << endl;
        }

        file.flush();

        file.close();
        return true;
    }

    void dA(const User& user) {
        cout <<
"\n================================
================================
================\n";
        cout << "                    APPLICATION
#" << user.id << "\n";
        cout <<
"================================
================================
================\n";
        cout << "Name        : " << user.name <<
"\n";
        cout << "Email       : " << user.email <<
"\n";
        cout << "Phone       : " << user.phone <<
"\n";
        cout << "Age         : " << user.age <<
"\n";
        cout << "\n--- Preference Scores ---\n";
        cout << "Cleanliness    : " <<
user.cleanliness << "/10\n";
        cout << "Sleep Schedule : " <<
user.sleepSchedule << "/10\n";
        cout << "Study Habits   : " <<
user.studyHabits << "/10\n";
        cout << "Social Level   : " <<
user.socialLevel << "/10\n";
        cout << "\n--- Description ---\n";
        cout << user.description << "\n";
        cout <<
"================================
================================
================\n";
    }

public:
    bool lg() {
        cout <<
"\n================================
================================
================\n";
        cout << "                ROOMMATE
FINDER - ADMIN LOGIN\n";
        cout <<
"================================
================================
================\n\n";

        string username, password;

        cout << "Username: ";
        getline(cin, username);
        cout << "Password: ";
        getline(cin, password);

        if (username == ADMIN_USERNAME
&& password == ADMIN_PASSWORD) {
            cout << "\nAdmin lg successful!\n";
            cout << "Welcome, Administrator!\n";
            pause();
            return true;
        } else {
            cout << "\nInvalid admin
credentials!\n";
            pause();
            return false;
        }
```

```
    }

    void rA() {
        ldUs();
        bPQ();

        if (pendingQueue.empty()) {
            cout << "\nNo pending applications
.\n";
            pause();
            return;
        }

        cout << "\n" << pendingQueue.size() <<
" application(s) are Pending \n";
        pause();

        int processed = 0;

        while (!pendingQueue.empty()) {
            User currentUser =
pendingQueue.front();
            pendingQueue.pop();

            dA(currentUser);

            cout << "\n[A] Approve | [R] Reject |
[S] Skip | [Q] Quit Review\n";
            cout << "\nYour decision: ";

            string input;
            getline(cin, input);
            char choice = input.empty() ? 'S' :
toupper(input[0]);

            if (choice == 'A') {
                if (uUS(currentUser.id,
"APPROVED")) {
                    cout << "\nApplication #" <<
currentUser.id << " APPROVED!\n";
                    cout << "User " <<
currentUser.name << " can now lg.\n";
                    processed++;
                } else {
                    cout << "\nError updating
status!\n";
                }
                pause();

            } else if (choice == 'R') {
                if (uUS(currentUser.id,
"REJECTED")) {
                    cout << "\nApplication #" <<
currentUser.id << " REJECTED!\n";
                    processed++;
                } else {
                    cout << "\nError updating
status!\n";
                }
                pause();

            } else if (choice == 'S') {
                cout << "\nApplication skipped.
Moving to next.\n";
                pause();
                continue;

            } else if (choice == 'Q') {
                cout << "\nExiting review mode.\n";
                break;
            } else {
                cout << "\nInvalid choice!\n";
                pause();
            }
        }

        cout << "\nReview session complete!\n";
        cout << "Processed: " << processed << "
application(s)\n";
        pause();
    }

    void vUBS() {
        ldUs();

        cout <<
"\n==============================
==============================
==========\n";
        cout << "                    ALL USERS BY
STATUS\n";
        cout <<
"==============================
==============================
==========\n\n";
```

```
        int pending = 0, approved = 0, rejected =
0;

        cout << "--- PENDING APPLICATIONS
---\n";
        for (const User& user : allUsers) {
            if (user.status == "PENDING") {
                cout << "ID:" << user.id << " | " <<
left << setw(25) << user.name
                     << " | " << user.email << "\n";
                pending++;
            }
        }
        if (pending == 0) cout << "None\n";

        cout << "\n--- APPROVED USERS ---
\n";
        for (const User& user : allUsers) {
            if (user.status == "APPROVED") {
                cout << "ID:" << user.id << " | " <<
left << setw(25) << user.name
                     << " | " << user.email << "\n";
                approved++;
            }
        }
        if (approved == 0) cout << "None\n";

        cout << "\n--- REJECTED
APPLICATIONS ---\n";
        for (const User& user : allUsers) {
            if (user.status == "REJECTED") {
                cout << "ID:" << user.id << " | " <<
left << setw(25) << user.name
                     << " | " << user.email << "\n";
                rejected++;
            }
        }
        if (rejected == 0) cout << "None\n";

        cout <<
"\n================================
================================
================\n";
        cout << "Total: " << allUsers.size()
             << " (Pending: " << pending
             << ", Approved: " << approved
             << ", Rejected: " << rejected << ")\n";
        cout <<
"================================
================================
================\n";

        pause();
    }

    void dS() {
        ldUs();

        int pending = 0, approved = 0, rejected =
0;
        double avgAge = 0;

        for (const User& user : allUsers) {
            if (user.status == "PENDING")
pending++;
            else if (user.status == "APPROVED")
approved++;
            else if (user.status == "REJECTED")
rejected++;

            avgAge += user.age;
        }

        if (!allUsers.empty()) {
            avgAge /= allUsers.size();
        }

        cout <<
"\n================================
================================
================\n";
        cout << "                    SYSTEM
STATISTICS\n";
        cout <<
"================================
================================
================\n\n";
        cout << "Total Registrations : " <<
allUsers.size() << "\n";
        cout << "Pending Queue Size  : " <<
pending << " \n";
        cout << "Approved Users      : " <<
approved << "\n";
        cout << "Rejected Apps       : " <<
rejected << "\n";
```

```
        cout << "\nAverage Age         : " << fixed
<< setprecision(1) << avgAge << " years\n";
        cout <<
"================================
================================
===============\n";

        pause();
    }
};

class MatchManagementModule {
private:
    vector<User> allUsers;
    vector<MatchRequest> matchRequests;
    vector<ConfirmedMatch>
confirmedMatches;
    map<int, vector<pair<int, double>>>
compatibilityGraph;

    void ldUs() {
        allUsers.clear();
        ifstream file(USERS_FILE);
        if (!file.is_open()) return;

        string line;
        while (getline(file, line)) {
            if (line.empty()) continue;

            stringstream ss(line);
            string token;
            vector<string> tokens;

            while (getline(ss, token, '|')) {
                tokens.push_back(trim(token));
            }

            if (tokens.size() >= 12) {
                try {
                    User user;
                    user.id = stoi(tokens[0]);
                    user.name = tokens[1];
                    user.email = tokens[2];
                    user.phone = tokens[3];
                    user.password = tokens[4];
                    user.age = stoi(tokens[5]);
                    user.cleanliness = stoi(tokens[6]);
                    user.sleepSchedule =
stoi(tokens[7]);
                    user.studyHabits =
stoi(tokens[8]);
                    user.socialLevel = stoi(tokens[9]);
                    user.description = tokens[10];
                    user.status = tokens[11];

                    if (user.status == "APPROVED")
{
                        allUsers.push_back(user);
                    }
                } catch (...) {
                    continue;
                }
            }
        }

        file.close();
    }

    void ldMR() {
        matchRequests.clear();
        ifstream
file(MATCH_REQUESTS_FILE);
        if (!file.is_open()) return;

        string line;
        while (getline(file, line)) {
            if (line.empty()) continue;

            stringstream ss(line);
            string token;
            vector<string> tokens;

            while (getline(ss, token, '|')) {
                tokens.push_back(trim(token));
            }

            if (tokens.size() >= 5) {
                try {
                    MatchRequest req;
                    req.senderId = stoi(tokens[0]);
                    req.receiverId = stoi(tokens[1]);
                    req.status = tokens[2];
                    req.senderName = tokens[3];
                    req.receiverName = tokens[4];
```

```
            matchRequests.push_back(req);
        } catch (...) {
            continue;
        }
    }
}

file.close();
}

void ldCM() {
confirmedMatches.clear();
ifstream file(MATCHES_FILE);
if (!file.is_open()) return;

string line;
while (getline(file, line)) {
if (line.empty()) continue;

stringstream ss(line);
string token;
vector<string> tokens;

while (getline(ss, token, '|')) {
tokens.push_back(trim(token));
}

if (tokens.size() >= 3) {
try {
ConfirmedMatch match;
match.user1Id = stoi(tokens[0]);
match.user2Id = stoi(tokens[1]);
match.matchDate = tokens[2];


confirmedMatches.push_back(match);
} catch (...) {
continue;
}
}
}

file.close();
}

double cC(const User& u1, const User& u2)
{
double score = 0;

score += (10 - abs(u1.cleanliness -
u2.cleanliness)) * 2.5;
score += (10 - abs(u1.sleepSchedule -
u2.sleepSchedule)) * 2.0;
score += (10 - abs(u1.studyHabits -
u2.studyHabits)) * 2.0;
score += (10 - abs(u1.socialLevel -
u2.socialLevel)) * 3.5;

return score;
}

void bCG() {
compatibilityGraph.clear();

for (int i = 0; i < allUsers.size(); i++) {
for (int j = i + 1; j < allUsers.size();
j++) {
double score = cC(allUsers[i],
allUsers[j]);


compatibilityGraph[allUsers[i].id].push_back(
{allUsers[j].id, score});

compatibilityGraph[allUsers[j].id].push_back(
{allUsers[i].id, score});
}
}
}

User* gU(int id) {
for (User& user : allUsers) {
if (user.id == id) return &user;
}
return nullptr;
}

static bool compareScores(const pair<int,
double>& a, const pair<int, double>& b) {
return a.second > b.second;
}

public:
void dCG() {
ldUs();
bCG();
```

```
        if (allUsers.empty()) {
            cout << "\nNo approved users to build
graph.\n";
            pause();
            return;
        }

        cout <<
"\n================================
================================
================\n";
        cout << "
COMPATIBILITY GRAPH \n";
        cout <<
"================================
================================
================\n\n";
        cout << "Graph: " << allUsers.size() << "
nodes,"
             << "Each edge = compatibility
score\n\n";

        for (const User& user : allUsers) {
            cout << "User " << user.id << " (" <<
user.name << ") -> ";

            if (compatibilityGraph.find(user.id) !=
compatibilityGraph.end()) {
                const auto& connections =
compatibilityGraph[user.id];

                vector<pair<int, double>>
topConnections = connections;

                sort(topConnections.begin(),
topConnections.end(), compareScores);

                int shown = 0;
                for (const auto& conn :
topConnections) {
                    if (shown >= 3) break;

                    User* connectedUser =
gU(conn.first);
                    if (connectedUser) {
                        cout << "[" << conn.first << ":"
<< connectedUser->name
                             << " (" << fixed <<
setprecision(1) << conn.second << "%)] ";
                        shown++;
                    }
                }
            }
            cout << "\n";
        }

        pause();
    }

    void vMR() {
        ldUs();
        ldMR();

        if (matchRequests.empty()) {
            cout << "\nNo match requests
found.\n";
            pause();
            return;
        }

        cout <<
"\n================================
================================
================\n";
        cout << "                    ALL MATCH
REQUESTS\n";
        cout <<
"================================
================================
================\n\n";

        map<string, int> statusCount;

        for (const auto& req : matchRequests) {
            cout << req.senderName << " (ID:" <<
req.senderId << ")"
                 << " -> " << req.receiverName << "
(ID:" << req.receiverId << ")"
                 << " [" << req.status << "]\n";

            statusCount[req.status]++;
        }

        cout <<
"\n================================
```

```
==========
==========\n";
        cout << "Total Requests: " <<
matchRequests.size() << "\n";
        cout << "Pending: " <<
statusCount["PENDING"]
             << " | Accepted: " <<
statusCount["ACCEPTED"]
             << " | Rejected: " <<
statusCount["REJECTED"] << "\n";
        cout <<
"==========
==========
==========\n";

        pause();
    }

    void vCM() {
        ldUs();
        ldCM();

        if (confirmedMatches.empty()) {
            cout << "\nNo confirmed roommate
matches yet.\n";
            pause();
            return;
        }

        cout <<
"\n==========
==========
==========\n";
        cout << "                CONFIRMED
ROOMMATE MATCHES\n";
        cout <<
"==========
==========
==========\n\n";

        for (const auto& match :
confirmedMatches) {
            User* user1 = gU(match.user1Id);
            User* user2 = gU(match.user2Id);

            if (user1 && user2) {
                cout << user1->name << " (ID:" <<
match.user1Id << ")"
                     << " <-> " << user2->name << "
(ID:" << match.user2Id << ")"
                     << " | Date: " <<
match.matchDate << "\n";
            }
        }

        cout <<
"\n==========
==========
==========\n";
        cout << "Total Confirmed Matches: " <<
confirmedMatches.size() << "\n";
        cout <<
"==========
==========
==========\n";

        pause();
    }

    void dMS() {
        ldUs();
        ldMR();
        ldCM();
        bCG();

        cout <<
"\n==========
==========
==========\n";
        cout << "                MATCHING
SYSTEM STATISTICS\n";
        cout <<
"==========
==========
==========\n\n";

        cout << "--- Graph Statistics ---\n";
        cout << "Total Nodes (Users): " <<
allUsers.size() << "\n";

        int totalEdges = 0;
        double avgCompatibility = 0;
        int edgeCount = 0;

        for (const auto& pair :
compatibilityGraph) {
```

```
            totalEdges += pair.second.size();
            for (const auto& edge : pair.second) {
                avgCompatibility += edge.second;
                edgeCount++;
            }
        }

        totalEdges /= 2;
        if (edgeCount > 0) avgCompatibility /=
edgeCount;

        cout << "Total Edges: " << totalEdges <<
"\n";
        cout << "Average Compatibility: " <<
fixed << setprecision(1)
             << avgCompatibility << "%\n";

        cout << "\n--- Match Request Statistics --
-\n";
        cout << "Total Requests: " <<
matchRequests.size() << "\n";

        int pending = 0, accepted = 0, rejected =
0;
        for (const auto& req : matchRequests) {
            if (req.status == "PENDING")
pending++;
            else if (req.status == "ACCEPTED")
accepted++;
            else if (req.status == "REJECTED")
rejected++;
        }

        cout << "Pending: " << pending << "\n";
        cout << "Accepted: " << accepted <<
"\n";
        cout << "Rejected: " << rejected << "\n";

        cout << "\n--- Confirmed Matches ---\n";
        cout << "Total Roommate Pairs: " <<
confirmedMatches.size() << "\n";

        cout <<
"\n==============================
==============================
================\n";
        pause();
    }
};

int main() {
    AdminApprovalModule adminApproval;
    MatchManagementModule
matchManagement;

    if (!adminApproval.lg()) {
        return 0;
    }

    while (true) {
        cout <<
"\n==============================
==============================
================\n";
        cout << "                ROOMMATE
FINDER - ADMIN PORTAL\n";
        cout <<
"==============================
==============================
================\n\n";
        cout << "1. Review Pending Applications
\n";
        cout << "2. View All Users by Status\n";
        cout << "3. Display System Statistics\n";
        cout << "4. View Compatibility
Graph\n";
        cout << "5. View All Match Requests\n";
        cout << "6. View Confirmed Matches\n";
        cout << "7. Match Statistics\n";
        cout << "8. Logout\n";
        cout << "\nChoice: ";

        string choiceStr;
        getline(cin, choiceStr);

        if (!isValidInt(choiceStr)) {
            cout << "\nInvalid input!\n";
            pause();
            continue;
        }

        int choice = stoi(choiceStr);

        switch(choice) {
            case 1:
                adminApproval.rA();
```

```
            break;
        case 2:
            adminApproval.vUBS();
            break;
        case 3:
            adminApproval.dS();
            break;
        case 4:
            matchManagement.dCG();
            break;
        case 5:
            matchManagement.vMR();
            break;
        case 6:
            matchManagement.vCM();
            break;
        case 7:
            matchManagement.dMS();
            break;
        case 8:
            cout << "\nAdmin logged out!\n";
            return 0;
        default:
            cout << "\nInvalid choice!\n";
            pause();
    }
  }

  return 0;
}
```
